Prism Template Pipeline

Format findings, code architecture and the complete styling scheme: REDCap export to filled .prism, any phase, any graph.

Revision 2026-11-09 (3). Part A is unchanged. Part B is reorganised around three results of this session: (1) the **Zval chunk is a tagged tree** and colours are **named keys**, not fixed offsets; (2) a graph binary can hold **more than one Zval chunk** and the graph's own is the **last**; (3) every structure a graph needs is already in a Prism file: from one written participant column, columns are copied, filled and reformatted freely (B0). B15 is the value-to-appearance reference with cleaned Prism renderings. READ = measured from bytes, seen in output or in a screenshot; INFERRED = conclusion from READ facts.

PART A - THE PIPELINE

A1. Layout

```
PrismUpdate/
 block_split.py   raw export -> block structure            run once per export
 varcsv.py        blocks -> one CSV per measure
 scantemps.py     source .prism -> temps_<phase>.csv skeleton
 buildtemps.py    coretemp -> per-measure templates         run once per phase
 prism_fill.py    template + CSV -> filled .prism
 run.py           drives block_split -> varcsv -> prism_fill
 map_<phase>.csv  variable, template, output, skip, sheetbase
 temps_<phase>.csv output, title, ylabel, ymin, ymax, ystep, coltitle, sheet*
 labels_<phase>.csv event -> row label
 raw/  corestructure/  Data/<phase>/  temps/  outputs/<phase>/
```

A2. The two halves

	Template build	Data fill
script	buildtemps.py (+ prismprep tools, B14)	run.py -> varcsv.py + prism_fill.py
input	one coretemp + temps_<phase>.csv	raw export + map_<phase>.csv
touches data.bin	yes: axis doubles, strings, wrap flag, styling	no
frequency	once per phase, or when a title/axis/style changes	every export

A3. Command sequence for a new phase

```
python3 scantemps.py temps/RawtempX -o temps_phaseX.csv        # constants; add a BASELINES entry (B5)
python3 buildtemps.py --base temps/Coretemps/coretempX.prism --map temps_phaseX.csv --out temps/PhaseX [--dry]
# write labels_phaseX.csv (rows EXACTLY as in the template) and map_phaseX.csv
python3 run.py --name phaseX --raw-match <substr> --map map_phaseX.csv --blank-missing
```

A4. Config columns

File / column	Meaning
temps: title, ylabel	replace coretemp's strings verbatim, including leading spaces
temps: ymin/ymax/ystep	written to the 9 axis doubles; blank leaves coretemp's
temps: coltitle	pattern with {pid}; JSON titles AND the binary copies
temps: sheetdata/graph	document.json sheet names and each sheet.json title; overwritten at fill time (B11)
map: variable / template / skip / sheetbase	varcsv spec (fieldA+fieldB unions); path under temps/; events dropped; sheet stem
labels: event,label	REDCap event -> template row label

PART B - THE FORMAT

B0. Framework [new]

Part 1 - generating templates: everything comes from structures Prism already has

Before: take a template and copy bytes at remembered offsets, imitating what Prism wrote. **Now:** every structure a graph needs already exists in a Prism file; we **move, copy and rewrite** those structures, with the format understood well enough (Zval tree with named keys, fenID binding, per-series parts) that any graph can be generated from one starting file.

The minimum starting point is a .prism with a data table and a graph, and **one participant column whose plot object Prism has written** (in practice: that participant styled once, i.e. non-default). A column that exists only in JSON has no plot object in the graph binary, so there is nothing to copy or append; the written plot object of the one participant is what makes the column copyable.

step	what	how
1	copy the column into any number of columns	pp_add --like <participant> --count N (data-set JSON cloned, new uid / fenID, appended to the table and graph lists); pp_rows for the number of rows
2	give every copy a value	pp_data (grid by time slot x column; --keep to leave other columns)
3	materialise the copies as series	only Prism creates series objects (member entry, record, style object, XYPlot item); a file carrying the series count is built once as a placeholder (one Prism save) and every later file is filled and styled from it by script
4	reformat each copy independently	any combination of the edits below, all by script, on the written structures

Everything that can then be edited by script (each row points to its section):

area	editable	where
table	number of rows and columns, row labels, every value, blanks	pp_rows, pp_add, pp_data / prism_fill (B1)
columns and legend names	data-set titles (legend label follows the title), column order	data-set JSON, sheet dataSets (B1, B2)
binding	which column uses which plot object	fenID in the data-set JSON (B2)
sheet and graph names	data sheet, graph, analyses, Row statistics	document.json sheetAttributesMap + sheet.json title; prism_build / prism_fill _retitle (B11)
graph scaling	y axis min, max, step	axis doubles located by structure (prism_build.locate_axis, B5)
titles and axis labels	graph title, y label (length fields at -4 and -18)	string records (B4); title box width fixed by the baseline (B6)
legend layout	entry wrapping	flag bit 1 at -16 of each name record (B4, B6)
symbols	shape (1-60), size, border thickness	style object p / p+75 / p-134; f32 p+255; border f32 p+215 / p+219 (B10, B15, B19)
lines	pattern, type (step, arrows), thickness	style object p-128, p-26, f32 p+227 (B10, B15)
colour	fill, border, line, error bars; any RGB; per-role alpha (hide)	XYPlot item in the LAST Zval chunk + style colour cells, both places (B12)
copying a whole look	one series' full style form onto another existing series	overwrite its style object, keeping idx46, data range, stamp, legend (B16, ZT12)

Limits that remain: series objects come only from a Prism-built placeholder (step 3, B19) and Prism's rendering limit (217 series fine, 1,600 crashed).

Part 2 - appending a template across common measures

Given a set of graphs with known scaling parameters and titles, one generated template renders all of them under the same format: a parameter row per graph (title, y label, y-axis min / max / step, sheet name) is written into a copy of the template; everything else carries over unchanged. The title must fit the title box fixed in the baseline (B6); axis values are located by structure on every run and the build aborts on a mismatch (B5).

Part 1 produces one template per phase: the structure (rows = the phase's events, columns = its participants, plot objects, styling). Measures in a phase share that structure, so one template is **appended** to every measure: only the measure-specific parts change, and every export refills the data. This is Part A, seen through the framework.

stage	what changes per measure / per export	how
per measure (once per phase)	graph title, y-axis label, axis min / max / step, column-title pattern, sheet and graph names	temps_<phase>.csv -> buildtemps.py / prism_build.py: strings (B4), locate_axis (B5), _retitle (B11)
per measure (mapping)	which REDCap variable(s) feed the measure, events skipped, sheet stem	map_<phase>.csv (A4); labels_<phase>.csv maps events to the template rows
per export (every run)	all values, blanks for missing events, participants without data	run.py -> varcsv.py -> prism_fill.py (pure JSON + table CSV; data.bin untouched); prism_trim.py drops empty participant columns
combining	per-measure files gathered into one workbook, phases overlaid	merge / overlay steps of prism_update.sh: data sets re-keyed with document-wide fenIDs, never by position

Rule that keeps appension safe: match everything by participant id and event label, never by position (B9); a template carries the look, the export carries only data.

B1. File structure

Member	Holds
document.json	sheet lists and sheetAttributesMap (displayed sheet names); customColors (save noise)
data/sheets/<SUID>/sheet.json	table: uid, dataSets = ORDERED dataset UIDs (column order); title / isCustomTitle
data/tables/<TUID>/data.csv, content.json	cell grid; numberOfColumns, numberOfRows (both editable in JSON: pp_rows)
data/sets/<UID>.json	per column: uid, **fenID**, title, format, replicates[] (firstRow / lastRow)
graphs/<GUID>/sheet.json	inputDataSets (JSON mirror of the member list) and the graph title
graphs/<GUID>/data.bin	axes, member list, idx46 list, name records, style objects, legend, and one or more **Zval chunks**; the graph's own Zval is the **last** chunk (B12)
analyses/<AUID>/sheet.json	Row statistics sheet and its title

Grid layout: col 0 row label, col 1 X, then each dataset takes w = len(replicates) columns (y_se: value + SEM, SEM left empty). Row i of data.csv is row i of the plot; blank cells draw nothing.

B2. What binds what - slots and columns [rev]

Nothing in data.bin references a dataset UID. A data set binds to a graph series by **fenID** (READ: the fenID sits 34 bytes before each series name). The binary member list (chunk 0) is the authoritative series order; reordering inputDataSets alone changes drawing order only.

Slot model (READ, tests E5 and the 5 x 20 grid): a **slot** is a series in the graph binary (member-list entry, name record, style object, Zval XYPlot item). It carries the whole **look**. A **column** carries the **data** and the **name** (the legend label follows the JSON title). A column takes over a slot by carrying its fenID; swapping fenIDs between a new column and an existing participant moves the participant's slot to the new column. **One column per slot:** when ten columns carried the same fenID only the last was drawn and the owner gave the slot up. Columns whose fenID has no slot in the binary are built by Prism at load (in memory only) and drawn in the automatic palette.

B3. Chunk grammar

Size / item	Meaning
250 / 33	three shared style prototypes (+ pair); NOT the per-series style
587	per style-group object
25 / 71	point / boundary records (Prism regenerates them from the data)
chunk 0 (~19-23 kB)	axes, titles, legend, member list, start of the style objects
large mid chunk	remaining series; NOT relocatable (internal offsets)
Zval chunk(s)	zlib-compressed render state (B12); the last one is the graph's

``` does not delimit series. No chunk size recurs once per series (inventory of a 21-series file), so a series is not one chunk.

## B4. Strings and name records

2c XX | len uint32 | ascii text | 2d XX XX. The length is repeated as a uint16 18 bytes before the text; that is what renders. A series name record is a 602-byte block [text-440, text+162).

| Offset from text | Meaning | Write on rename? |
|---|---|---|
| -4 (u32), -18 (u16) | length (both) | YES |
| -16 (u16) | flags; bit 1 wraps the legend entry | clear bit 1 |
| -24 (u16), copy -238 | legend slot counter; must be unique | NO |
| -28 (u16) | constant 18; not a length | NO |
| -34 (u16) | **fenID** of the owning data set (binding) | NO |
| -46, -44 (u16) | **idx46**: the series' internal index (B13) | NO |

## B5. Axis values

min, max and step are doubles in chunk 0, in two clusters; locate by value structure (prism_build.locate_axis), never by remembered offsets. coretempII: MAXOFF 1952/1968/5163, MINOFF 1960/1976/5155/5215/5223, STEPOFF 5207 (3-15 step 2).

## B6-B8. Layout limits, ids, events
```

Title box width is fixed at save time (pad with spaces in the baseline). Legend wrap: clear bit 1 at -16. Phase II ids P###, Phase I M###; scripts match [PM]\d+. Events differ by phase (_arm_1 vs _arm_2/_arm_3); every phase needs its own labels file.

B9. Things that cost time - do not repeat [rev]

Assumption	Reality
The first Zval chunk is the graph's	A Prism-built file can hold two; the first had 0 XYPlot items, the last had all 217. Every edit to the first is invisible. Use the LAST (zlast.py).
Colour lives at fixed record offsets (807-byte records)	That was one serialisation. Zval is a tree; colours are named keys whose positions vary. Parse and write by name (zval.py).
Style-object colour cells alone set colour	They are the second place only. Colour must be written in BOTH places: the Zval XYPlot item and the style cells.
Inserting XYPlot items colours new columns	Items are handed out by position to the series in the binary; extra items shifted colours onto existing series (E2).
A default series stores default colour values	In a file not saved by Prism it has no colour entry at all; Prism falls back to its palette.
Keeping each item's alpha byte is harmless	Default open symbols carry fill alpha 00; a filled square then draws invisible. Set alpha ff for fills you want drawn.
Copying blocks into data.bin creates a series	E4 / E6 (member entry, idx46 entry, record, style object inserted) broke the file. Slot creation by insertion is not solved (B17).
Opening a file in Prism changes it	Prism builds new series in memory; it does not save an unchanged document. Script outputs must be complete on disk.
Checking a file in Prism writes it	No. Prism builds missing series in memory on open and does not save an unchanged document; use Save As to force the write (P0on_border_100: 1 style object on disk after viewing).
pp_add creates series	No. It clones data-set JSON only (table and graph lists); the graph binary keeps its slots. New columns get slots only from a Prism save (B2, B14).
Style-object fields sit at the B10 offsets in every file family	Colour cells do; later fields do not in the P0 family (size p+223, 6.0 at p+215 / +219 / +239), and the object grows with explicit colours (B19).
A slot belongs to a series name	No. It belongs to whichever data set carries its fenID (B2): reusing slots in order put P101's slot on the data set renamed P108.
Every measure fits the full 14-point axis	No. Lines join consecutive rows only; a measure on 5 of 14 visits needs its own axis (B19).
Series objects can be created by script	No. Prism creates a series' member entry, record, style object and XYPlot item only when it saves; script clones broke (E4 / E6) and JSON copies stay unstyled. For objects to have styles, the file must already carry the series count: build a placeholder once in Prism, then fill and style it by script (B19).
pp_data fill leaves other columns alone	--blank-missing blanks every column not in the grid; use --keep.
Locating style objects by a valid code byte	8 default objects carried codes 56/57/58 (auto cycle) and were missed; take every position on the 923-byte grid.
One-file diff after a GUI edit isolates the change	Every save rewrites stamps, legend coordinates and the Zval stream; diff two saves or filter them.
Copying the mid chunk between files	Crashes Prism: internal offsets.
Writing axis min / max / step into any core	With the axis range on Auto, Prism recomputes it on open and ignores the stored doubles (V2). Apply the on-patch (B18) first.
Writing a title's text renames it	Automatic titles (sheet name, column name, default) are redrawn from their source; the record flags must be switched to custom as well (B18).
Offsets read from one save hold for the next	Re-saving P0 moved the Y clusters by one byte (905 -> 904). Every write locates its field by value or pattern and asserts the current value first.

B10. Per-series style object - field map [rev]

READ: every series has a 923-byte style object; the objects are contiguous on a 923-byte grid, in reverse member order. Offsets relative to the symbol-code byte p; the object spans [p-380, p+543).

rel to p	type	field	renders?	values / notes
0, +75	u8 x2	**symbol type**	yes	1-60 (B15); base at -134 = code if code <= 13, else 1
+255	f32	**symbol size**	yes	mirrors u8 -132 / +47; sizes 1-10 render progressively larger (B15)
+215, +219	f32 x2	**symbol border thickness**	inferred	1/12 pt, same values as line thickness; index mirrors u8 -72 / +119 (index = 7 + w / 6); to confirm by render (B19, V7)
-128	u8	**line pattern**	yes	0 solid, 1 dashed, 2 dotted, 3 dash-dot, 4 dash-dot-dot (B15)
-26	u8	**line type**	yes	0 plain, 1-2 step variants, 3-8 arrows, 9+ plain; mirror -130
+227	f32	**line thickness**	yes	1/12 pt: -3 Auto (= 1/2 pt), 3, 6, 12, 18, 24, 30, 36, 42; index mirror -102 / +111

rel to p	type	field	renders?	values / notes
-106/+195, -110/+191, -118/+139/+199, -126/+171	RGB+flag	fill, edge, border, line colour	second place	must match the Zval XYPlot item (B12)
-328, -194, -30	u16	**idx46** (identity)	-	equals the series' entry in the idx46 list and its name record
+87, -175, -180	u8	explicit / automatic flags	-	explicit series 00/01/02, Prism-created defaults 01/00/00
-152, -144	f64 x2	cached data range	-	per series; keep
+151..+155	5 B	save stamp	-	shared per save group
-324..-280, +395..+417	u16s	legend layout	-	recomputed on save

B11. Sheet and graph names

document.json sheetAttributesMap and each sheet.json title; prism_build.py / prism_fill.py set data, graph, analyses and Row-statistics titles (installed live).

B12. The Zval chunk - tree grammar and named colour keys [rewritten]

Container (READ): c0 40 52 08 | u32 stream+16 | "Zval" | u32 stream | u32 decoded | zlib stream | "Zend" 12 40 01 40. A graph data.bin can hold **several** such chunks; only the **last** one has the graph's plot arrays (PCFF_GridLine_Array, PlotWithPoints_Array) and its XYPlot items.

Grammar (READ; parse -> rebuild is byte-identical on every file tested):

```
value := 'Mval' u32 type  u32 len  payload[len]  'Mend'
dict  := ( 'Dkey' u32 n  name[n]  'Dval' value )*  'Dfin'      (type 5)
array := 'Asiz' u32 count  ( 'Aval' value )*count  'Afin'     (type 6)
raw   := bytes                                               (type 1; a colour is 4 bytes R G B alpha)
```

Root keys: IPGAxis_Array, PCFF_Graph::bkgndcolor, PCFF_GridLine_Array, PlotWithPoints_Array, SEDObject_Array. **SEDObject_Array** items are dicts of five PCFF_*Style keys plus one of: **XYPlot_Map** (one per graph series: its colours), RowTitleList_Map (one per table column: formatting), or nothing (shared styles).

XYPlot_Map key	role	old fixed offset (one serialisation)
PCFF_Plot::connectlinecolor	line colour	+0
PCFF_Plot::symbolcolor	symbol fill	+274
PCFF_Plot::symbolBorderColor	symbol border	+220
PCFF_Plot::bordercolor	border (alpha 00)	-59
PCFF_Plot::errorcolor	error-bar colour (was called "edge")	+53
PCFF_Plot::fillcolor, patterncolor, BarBoxBordercolor	bar / box colours	-

Order (READ, E2): XYPlot items form one contiguous block (index 11 in the files tested, after the ".....rr.." prefix) and are handed out **by position** to the series in the binary member list, in reverse member order. Every XYPlot item has the same key layout and size; only the 4-byte values differ. Any RGB renders and survives a Prism save byte-exactly (X1: one-unit neighbours such as fe0000 / ff0000), i.e. beyond the GUI picker.

B13. Per-series parts and lists [new]

part	where	identity fields
member-list entry, 12 B	list @746 (stride 12); count u16 @744	fenID (u16) + fixed tail ff..
idx46 list entry, 4 B	list @1788; count u32 @1784; byte length u32 @1780 (= 4 + 4n)	idx46
name record, 602 B	per series (B4)	fenID -34, idx46 -46/-44, slot -24/-238, name text
style object, 923 B	contiguous grid (B10)	idx46 -328/-194/-30
XYPlot item	last Zval chunk, block in reverse member order	position
companion object (1,020 B, keyed by idx46)	not located in the test files	idx46

Offsets of the lists are coretempII-family values; locate by structure (count == run length) before writing.

B14. Building any graph - the recipe and the tools [new]

All tools live in ~/D/pwork/prismprep (new) and ~/D/pwork/tools (pstyle, pcolor, pwrite). They write new files only; inputs are never modified.

tool	does
pp_rows.py SRC DST N [--like P101]	resize the participant table to N rows labelled 1..N, values cleared (JSON / CSV)
pp_add.py SRC DST --like P101 --count N	add N columns (P201..) by cloning a data set JSON: new uid, document-wide next fenID, appended to dataSets and every inputDataSets, table widened
pp_data.py SRC DST GRID [--keep]	grid by time slot x series -> prism_fill; --keep leaves columns not in the grid untouched
fenID swap (inline, hue_ref.py: bind)	bind new column k to existing slot k by swapping fenIDs in the two data-set JSONs
pp_style.py SRC DST SPEC.csv / pwrite.Style	symbol, size, pattern, type, thickness, colours and alphas per series (both colour places)
zval.py	parse / build the Zval tree (lengths recomputed); walk() for paths
zlast.py	split / join the LAST Zval chunk (verified by its Zend trailer)
hue_ref.py OUTDIR [--slots N] [--scheme hue|helix|bands]	the 20-slot references (points and lines) by the recipe
grid_template.prism (+ inline ZT13-ZT16)	217-series template built by Prism once; colours written in the last chunk + style cells

Recipe 1 - up to the number of explicit slots (no Prism step)

```
pp_add  (columns)  ->  pp_data  (values; --keep to retain others)  ->  bind (fenID swap)  ->  style (pwrite: both colour place
```

Each slot shows one colour; a column can carry a point at every row, so the point count is unlimited but distinct looks = slots (20 participants + mean in the coretemp family).

Recipe 2 - many distinct colours (one Prism-built template)

```
pp_rows (N rows) -> pp_add (M columns) -> fill a value in every new column -> open + save in Prism ONCE (the placeholder: buil
-> per series: XYPlot item colours in the LAST Zval chunk + style colour cells (reverse member order) -> uniform symbol/size -
```

The template is reusable for every graph up to M series: values, colours and symbols are rewritten by script. 217 series rendered; 1,600 crashed Prism while rendering (B17).

B16. Reformatting guide - what to write where

change	write	notes
symbol shape	style object p and p+75; base p-134	codes B15; borderless squares/circles vanish when white on white
symbol size	f32 p+255; u8 p-132 / p+47	sizes 1-10 shown in B15
symbol border thickness	f32 p+215 and p+219; u8 index p-72 / p+119	inferred (B19); a thicker border also makes the symbol look larger
colour (fill, border, line)	XYPlot_Map symbolcolor / symbolBorderColor / connectlinecolor in the LAST Zval chunk AND style cells	keep or set the alpha byte deliberately (ff opaque, 00 hidden)
hide a role	alpha 00 on that role (pwrite.alpha)	symbols hidden: code 60 + size 1 + alpha 00 on fill/edge/border
line pattern / type / thickness	p-128 / p-26 (+ -130) / f32 p+227 (+ index -102, +111)	make lines visible first (pattern 0, width >= 12)
which column a look belongs to	data-set JSON fenID (bind)	one column per slot
table size, values, names	pp_rows / pp_data / data-set titles	legend label follows the JSON title
axis range, titles, sheet names	prism_build (locate_axis, strings, _retitle)	B5, B4, B11

B15. Value to appearance reference [rev]

Every figure is Prism's own rendering of a file generated by script from the coretempII baseline; the value written is the only thing that varies across the series shown. Unless stated otherwise the series are the 20 participants in pid order (P101 ... P123), drawn as flat lines or rows at equal heights so that only the swept property differs.

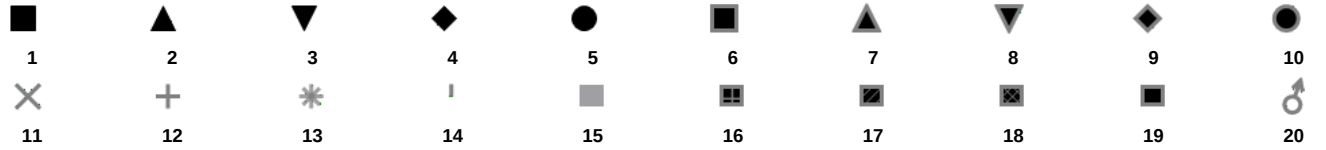


Figure 1. Symbol type, codes 1-20 (u8 at p and p+75; base p-134)

Shown: 1-5 filled square, triangle up, triangle down, diamond, circle; 6-10 the same shapes bordered; 11-14 cross, plus, asterisk, tick; 15-20 square variants and male sign. How determined: three reference files (R01-R03), each writing 20 consecutive codes to the 20 participants in pid order (code at p and p+75, base = code if code <= 13 else 1), size 6, lines hidden (line alpha 00), fill black, edge and border grey; tiles cut from Prism's legend, one per entry; tile number = code written. Image processing: plot-area grid removed (pixels on the green-to-white blend of the Prism grid set to white; grid pixels touching a mark reset where they were not part of a solid mark), then trimmed to content.

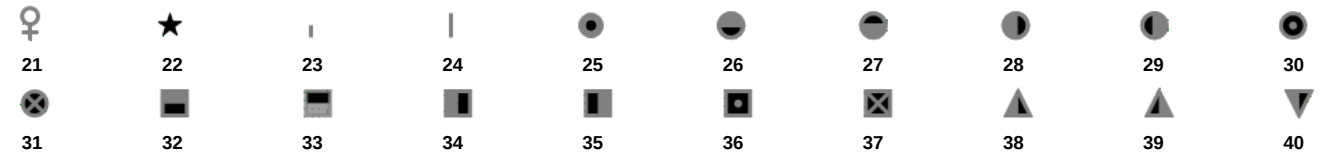


Figure 2. Symbol type, codes 21-40

Shown: female sign, star, ticks and bars, then circle, square and triangle two-tone variants (dot, halves, cross); the second tone is the grey edge colour. Determined as in Figure 1 (reference file R02).

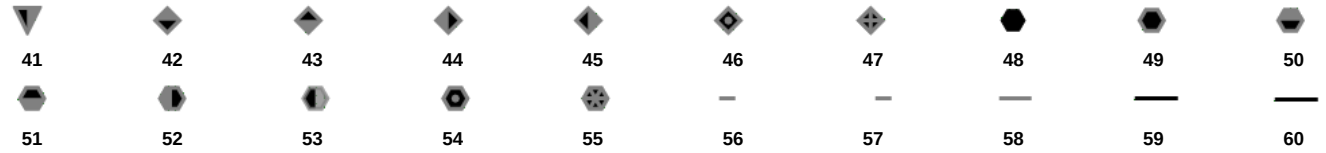


Figure 3. Symbol type, codes 41-60

Shown: triangle and diamond two-tone variants, hexagons (filled, halves, ring, dotted), 56-58 dash / tick marks, 59-60 no symbol (the tile shows the legend's line sample). Determined as in Figure 1 (reference file R03).



Figure 4. Symbol size 1-10 (f32 at p+255; integer mirrors p-132, p+47)

Shown: a filled square (code 1) at each size, tile number = size written. How determined: reference file R04 wrote sizes 1, 2, 3 ... to the participants in pid order, with the integer mirrors set to the same value; lines hidden; tiles cut from the legend (P101-P113). Each step draws larger through 10. Image processing: plot-area grid removed (pixels on the green-to-white blend of the Prism grid set to white; grid pixels touching a mark reset where they were not part of a solid mark), then trimmed to content.

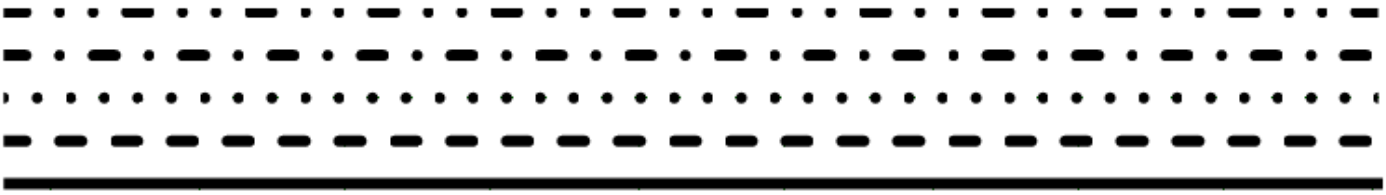


Figure 5. Line pattern (u8 at p-128)

Shown, bottom to top: 0 solid, 1 dashed, 2 dotted, 3 dash-dot, 4 dash-dot-dot. How determined: reference file R05 swept the pattern byte 0-19 across the participants; every line 1 pt (thickness 12), straight, black; symbols hidden (code 60, size 1, fill / edge / border alpha 00); flat data at equal heights so only the pattern differs. Codes 5-19 produced no further variants at this width. Image processing: plot-area grid removed (pixels on the green-to-white blend of the Prism grid set to white; grid pixels touching a mark reset where they were not part of a solid mark), then trimmed to content.

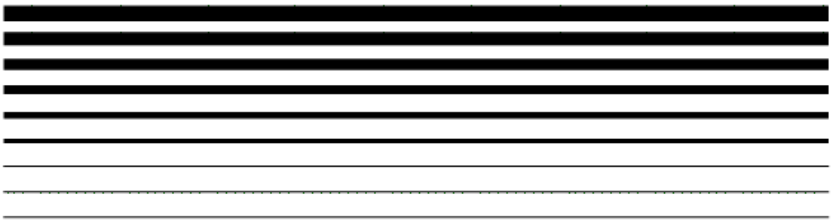


Figure 6. Line thickness (f32 at p+227; index mirrors p-102, p+111)

Shown: the stored widths, thickest at the top. How determined: reference file R07 cycled the thickness options across the participants: -3 (Auto, draws as 1/2 pt) and 3, 6, 12, 18, 24, 30, 36, 42 in units of 1/12 pt (1/4 pt to 3 1/2 pt), with the matching option index in the mirrors; pattern 0, symbols hidden, flat data. Values above 42 draw thinner than 42. Image processing: plot-area grid removed (pixels on the green-to-white blend of the Prism grid set to white; grid pixels touching a mark reset where they were not part of a solid mark), then trimmed to content.

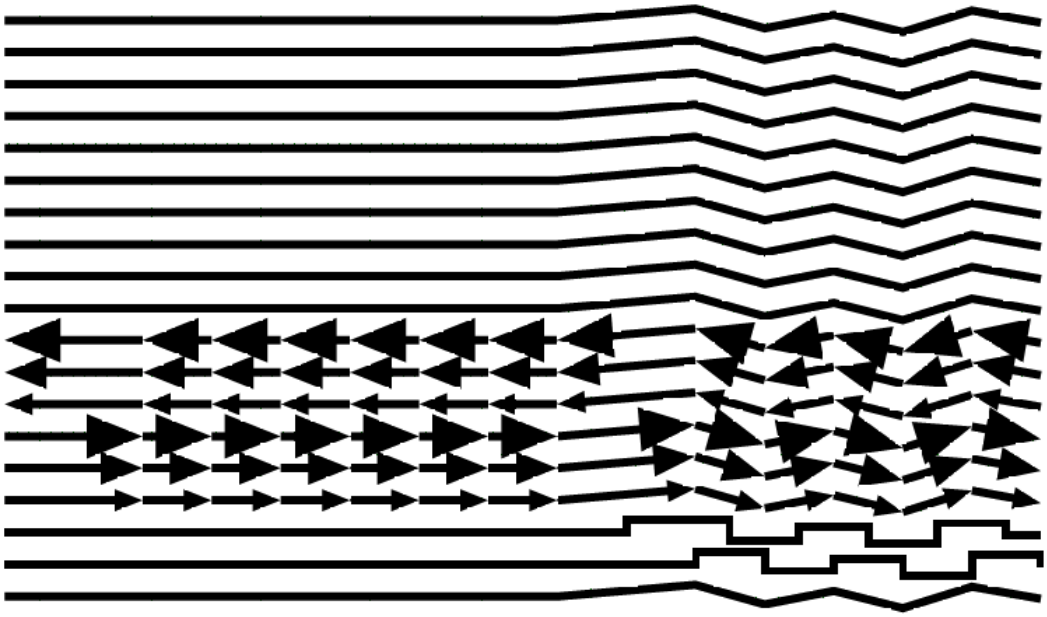


Figure 7. Line type (u8 at p-26; mirror p-130)

Shown, bottom to top: type 0 plain, 1 and 2 step variants, 3-8 arrows (three forward sizes, three backward), 9-19 plain. How determined: reference file R06 swept the type byte 0-19 across the participants; data flat for the first half of the time points and jagged for the second, so step and arrow behaviour shows on both kinds of segment; symbols hidden. Image processing: plot-area grid removed (pixels on the green-to-white blend of the Prism grid set to white; grid pixels touching a mark reset where they were not part of a solid mark), then trimmed to content.

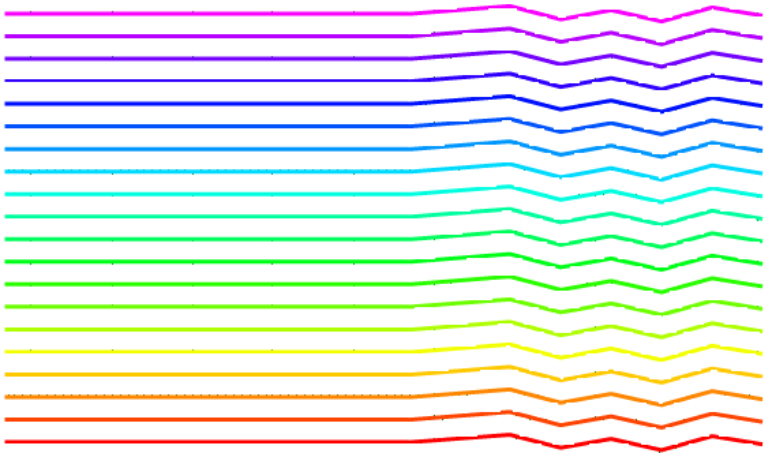


Figure 8. Line colour: one colour along the whole line (HUE20_lines)

Shown: 20 lines, hue red to violet, each keeping its colour through every change in amplitude. How determined: hue_ref.py by the template recipe: 20 columns P201-P220 added (pp_add), filled over 14 time points, flat then jagged (pp_data), bound to the 20 participant slots by fenID swap, then styled: colour hsv(0.83 k/19, 1, 1) written in both places (XYPlot item and style cells), 1 pt solid straight, symbols hidden. Image processing: plot-area grid removed (pixels on the green-to-white blend of the Prism grid set to white; grid pixels touching a mark reset where they were not part of a solid mark), then trimmed to content.

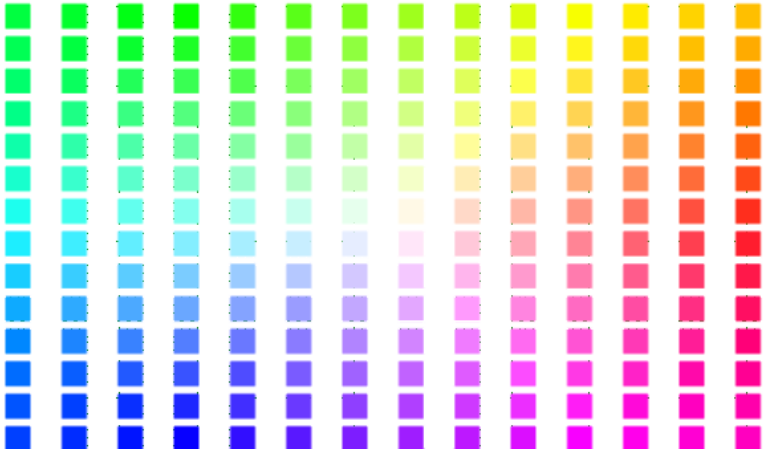


Figure 9. Symbol colour: the colour plane, every point its own series (ZT16)

Shown: 196 series on a 14 x 14 grid, hue = angle around the grid centre, saturation = distance from it (1.25 r / sqrt 2, capped at 1), value 1; the centre is white and is invisible on white without a border. How determined: template generation part 1: pp_rows (14 rows), pp_add (196 copies of P101's column), one value per column at time i mod 14 and height i div 14 (one Prism save: the placeholder step); then each series' colour written in the LAST Zval chunk (XYPlot symbolcolor / symbolBorderColor) and its style cells, series order = reverse member order; all symbols set to code 1, size 6; symbol alpha set to ff (default open symbols carried 00). Image processing: plot-area grid removed (pixels on the green-to-white blend of the Prism grid set to white; grid pixels touching a mark reset where they were not part of a solid mark), then trimmed to content.

B18. Construction from P0 - the generator launch point [new]

P0 is the minimal core every graph is generated from. It is made once, in Prism, with as few inputs as possible so that no editing state is cached: an **XY table in mean / SEM format** (y_se), the time points typed as row titles (blank rows as spacers, which space the labels on the x axis), one data column typed P101 (Prism auto-completed the name), the graph changed to a **line + dot** plot through the graph-type menu, everything else default. READ from P0: the row titles are their own text data set (rowTitlesDataSet); the X data set is empty, so x is the row index; the row labels are not stored in the graph at all (neither data.bin nor Zval) - the x-axis labels are read from the table at render time.

Method. Each property was found by one GUI change saved as the next file (P1, P2 ...), diffed against the previous one (diff_core.py: JSON keys, graph-binary byte runs decoded as int16 / int32 / float / double, Zval values by key path), then **verified** by writing the field by script into a copy of P0 and opening it (V files). Every write asserts the current value first and locates fields by value or byte pattern, never by a remembered offset.

file	GUI change (diffed against the previous file)	what it isolated
P1 / P9	X axis 8.5 -> 13.5 cm (P1 did not register; redone as P9)	X length u16 (two copies), plot-rect right edge, tick-label positions
P2	stretch Y axis	Y length u16 (two copies), plot-rect top edge
P3	move the graph 5 arrow taps	one tap = 6 units; every element coordinate shifts by the same offset
P4 / P5 / P6	Y / graph / X title typed	string record text + lengths + four flag bytes
P7	bordered symbol, transparent fill, blue border and line	symbol code pair + base; XYPlot colours (fill alpha 00)
P8	line pattern dotted	pattern byte = code byte - 128 (GUI "dotted" wrote 1)
P10 - P13	graph pushed to the page top, top-left, bottom-left, bottom-right	page bounds and margins (P10 unreadable: structure shifted, set aside)
P0on	every axis-format toggle set to manual / custom, values unchanged	the on-patch (below)

The on-patch. P0on differs from P0 in 54 edits (toggle bytes 00 -> 0f / 80 / 03, a small inserted block for the custom origin, save stamps). make_on_patch.py records them as P0_on.patch.json and applies them to P0 offset-correctly (inserts included); the result equals P0on byte for byte before Zval and in every Zval value. **Every generated file starts as P0 + on-patch**, so P0 is the only file the generator needs.

Units and layout. Layout values are u16 in **1/600 inch**: 236.22 units per cm (X 8.5 -> 13.5 cm read 2014 -> 3188). The plot rectangle (axes frame) is four u16 (left, top, right, bottom) with right - left = X length and bottom - top = Y length. Changing an axis length changes the plot rectangle and the graph box 1:1; the margins stay fixed for given content: left 1.52 cm (Y labels and title) and top 1.25 cm (graph title) measured; right 3.12 cm (legend) and bottom 3.17 cm (slanted time labels, X title) derived for a US Letter page (21.59 x 27.94 cm, to confirm). So graph box = (X + 4.64 cm) x (Y + 4.42 cm); the margins change with label, title and legend content and are measured once per core layout. Position is one offset applied to every element; it is left as saved in the core.

write	fields	verified
time axis: names and number of rows	table only: row-title data set text, numberOfRows, data.csv, replicate extents (the data.dt checksum need not be updated)	V1a, V1b
Y range and step	B5 doubles: cluster 1 [min, max, min], cluster 2 [min, max] with step at +52, located by value; requires the on-patch	V2
X / Y axis length	u16 length and its copy (X: P0 offsets 864 / 3036; Y: 930 / 4127); Prism recomputes ticks, plot rectangle and layout	V3, V4
titles (graph, Y, X)	string record text + u32 length (t-4) + u16 length (t-18); flags: t-15 -> 84; t-413 / t-367 / t-189 set 0x40, clear 0x08; t-414 / t-368 / t-190 clear 0x04	V5a-c
symbol, pattern, colours	style object located by its code pair (code at p and p+75, base p-134; P0 anchor 6899): code, base, pattern p-128; XYPlot colours in the last Zval chunk	V6

Default and adjustable mode. The default mode is this project's format (the 14 time points with two spacer rows, the axis and title conventions of the measures); the adjustable mode exposes the same writes for any other table shape that fits the generator, starting from the same P0. Series come from a placeholder that carries the series count (B19), since only Prism creates series objects. Not yet measured: the legend margin as a function of N.

B19. Template generator: designer JSON + core templates [rev]

Architecture (final). The look of a graph is a JSON file made in the designer (designer.html): per-series symbol code, fill, border and line colour, size, border thickness, line type, pattern and thickness, plus the mean's own block. **The JSON is the template.** It is applied to a **core template** (a PrismBreak coretemp: Prism-built, one styled slot per participant plus the mean, in the byte layout pwrite / pp_style are verified on, B10 / B12): design2spec.py turns JSON + core template into a pp_style SPEC (participants in id order take the designer's series 1..N; the mean takes the Mean block) and runs pp_style; update.py then fills the styled templates as before. The number of styled participants is capped by the template's slots (20 per phase, 40 + 2 means per overlay).

piece	location	role
designer	designer/designer.html	offline page; preview at P0 geometry with the graph's title, scale and data (designer_ctx.js); Confirm downloads the JSON
design launcher	tools/design.py	writes the preview context for a graph, opens the designer, waits for the JSON, saves it to config/designs/<name>.json
converter	tools/design2spec.py	design JSON + core template -> SPEC -> pp_style -> styled template
batch	tools/apply_design.py	every core template of a phase (config/graphs.csv) styled from one design into variants/<design>/; then filled by update.py
measure reference	config/measures_ref.csv (build_ref.py)	titles from the REDCap dictionary labels; data-driven Y scale (below)

Designer. Steps: symbols (fixed set, all filled, all bordered, random from a toggleable pool; tiles are the real Prism renders recoloured), colour, size and symbol border thickness, lines (type, pattern, thickness as drawn samples), mean (symbol, fill and border pickers, size, border thickness, lines, on / off), confirm. Rendering: tile grids and strips are built once and repainted in place; one render per animation frame; only the open step and the preview redraw; the wheel ring is cached.

Colour model. Series i = 10g + k lies in group g (groups of 10; 2 groups for 20 series, at most 10). Fill hue = base + 0.37 floor(i / 20) + shift (g mod 2) + smear (k / 9 - 0.5) / 3; brightness = top - 0.40 k / 9: each group a light-to-dark band, the two groups of each 20 shifted, each 20 far around the circle, smear fanning a group over up to 120 degrees (wheel ticks show the samples). Target all series (one scheme) or one group (its own hue, same band); while any group has its own hue the all-series option is locked until reset. Border: darker / lighter shade of the fill, same as fill, one picker colour, or a gradient from the picker colour with its own smear; groups can take their own border colour.

Measure reference. Titles written from the dictionary field label and form (e.g. "DERS-18: Awareness"); X title "Time"; Y scale from the data: [min - 0.5 SD, max + 0.5 SD] over all participants and time points, clamped to the instrument's published range, step = SD rounded to 1 / 2 / 2.5 / 5 x 10^k with 4-10 steps, min and max snapped outward; the statistics are stored beside each axis. 27 questionnaire measures; the 9 cognitive-battery files are set aside. A measure's time axis is the rows its data occupy: Prism draws a line only between consecutive rows that both have values, so a measure taken at 5 of 14 visits on the full axis shows almost no lines.

The P0 route (structure only). prismcore/tui/generate.py writes everything verified on P0-family files (B18: time axis, Y range and step on a base with the axis toggles manual, titles with custom-text flags, axis lengths) and saves the designer JSON into config/designs/. It does not style: in the P0 family the colour cells sit at B10's offsets, but the fields after them do not (P7 / P8: size f32 at p+223, not p+255; 6.0 at p+215 / +219 / +239, not p+227), and the object changes length when colours are explicit (P6 -> P7), so fixed-offset writes damage the file and pwrite correctly refuses it. A P0 base also has one slot only (B2); more slots need a Prism save (Recipe 2).

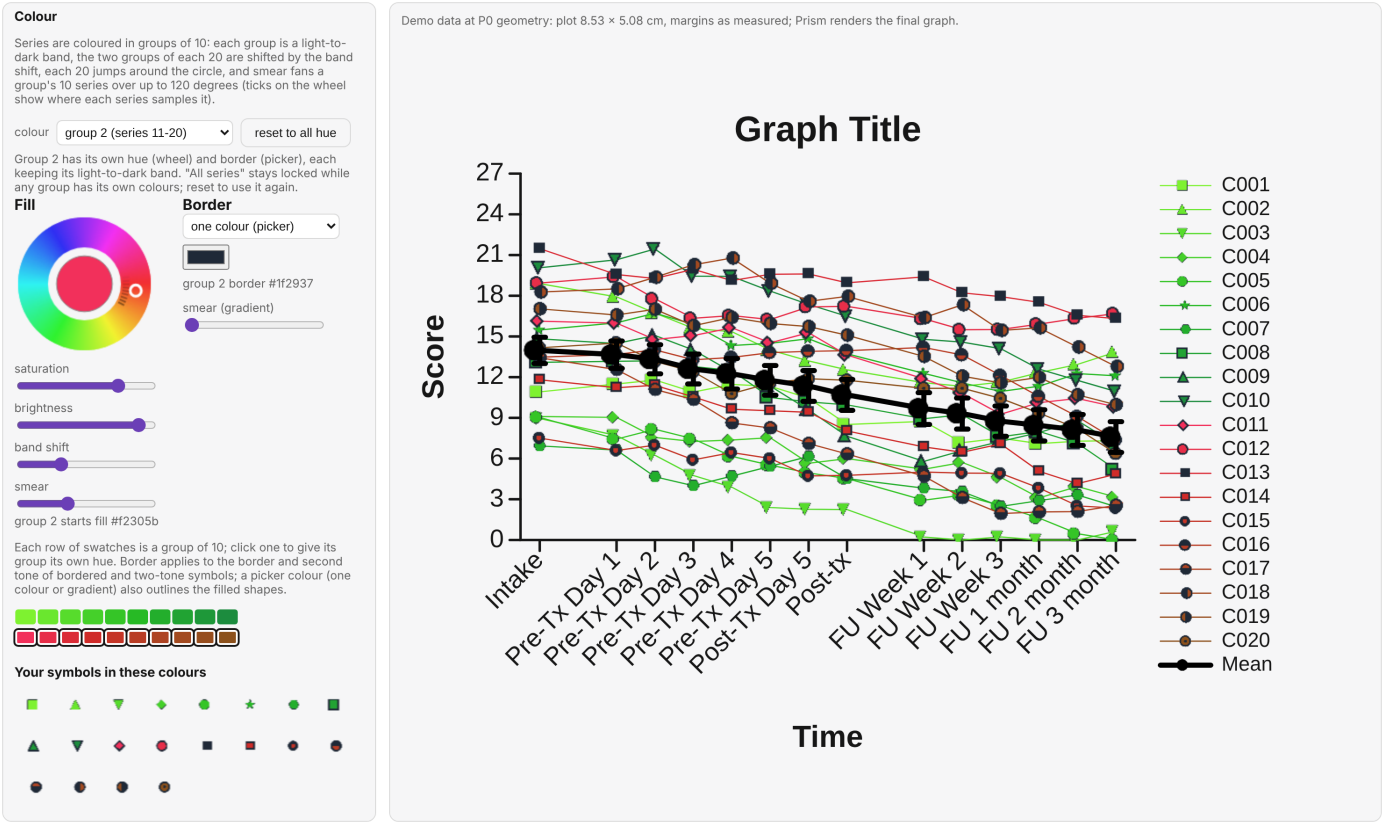


Figure 10. Designer, colour step: group 2 on its own hue; fill wheel with smear; border from the picker; preview at P0 geometry

Rendered headless from designer.html with demo data. Group 1 keeps the scheme hue with smear 0.35; group 2 (series 11-20) has its own hue; borders are one picker colour; the legend shows each series' line and symbol.

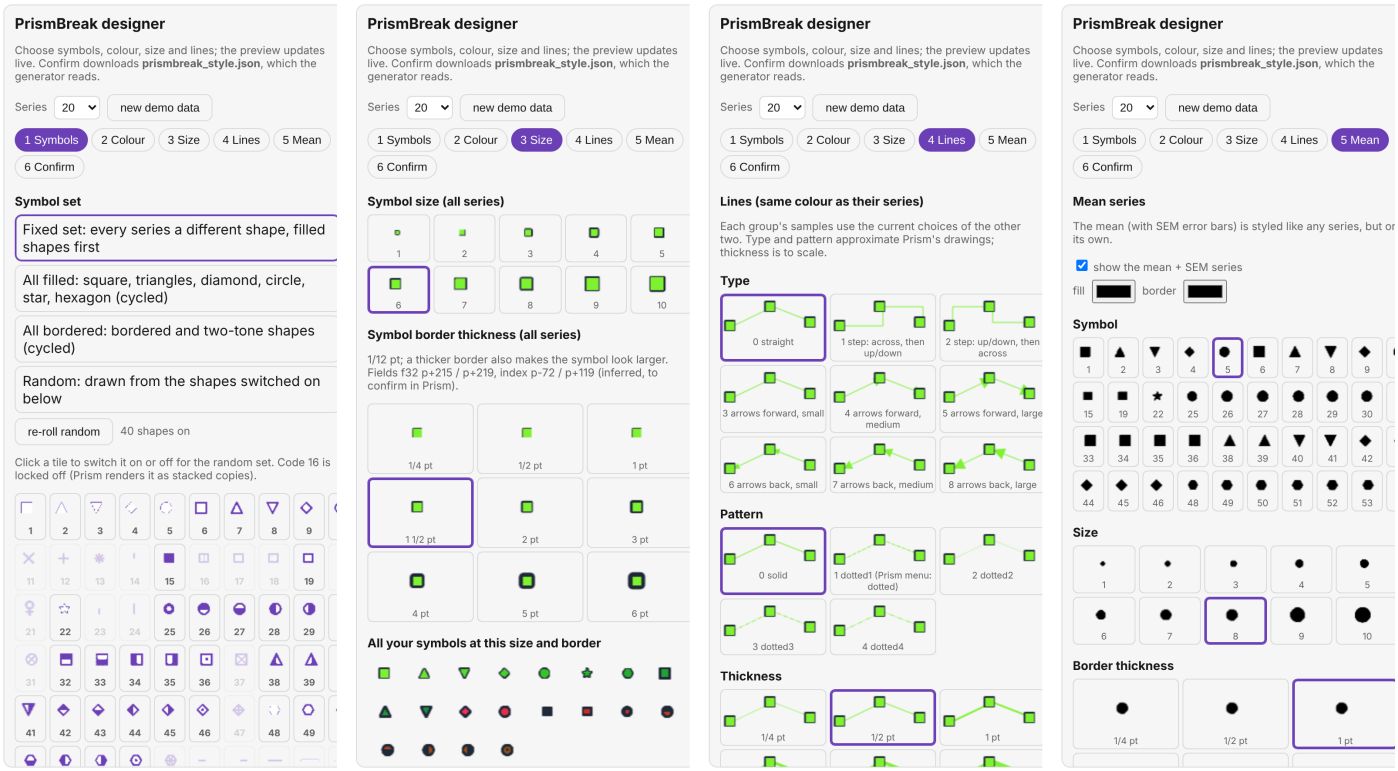


Figure 11. Designer, symbol, size, line and mean steps

Left: the symbol sets and the random pool (faded tiles are off; code 16 locked). Middle: size tiles and symbol border thickness tiles, drawn with series 1's symbol and colour. Third: line type, pattern (solid, dotted1-4) and thickness as drawn samples. Right: the mean's own symbol, colours, size and lines. Line drawings approximate Prism's (figures 6-7); thickness is drawn to scale.

B17. Open items

item	state / how to close it
design -> core template, first render	design2spec / apply_design written; the first styled core template confirms the mapping (participants in id order, mean block)
symbol border thickness	inferred f32 p+215 / p+219, index p-72 / p+119 (coretemp layout); apply_design writes it; one render confirms it
overlay means	owners() keys both overlay means as "mean", so only one is addressable; needs distinct keys
line thickness 4-6 pt	menu options above 3 pt; stored values (assumed 48 / 60 / 72) to confirm with one GUI change and diff
P0-family style layout	fields past the colour cells move (size p+223; 6.0 at p+215 / +219 / +239) and the object grows with explicit colours; unmapped, so the P0 route does not style
series objects by script	not achieved (E4 / E6); a Prism-made one-series diff would be the starting point; Recipe 2 (one save) is the working route
page size of P0	confirm (File > Page Setup) to fix the right and bottom margins of the P0 layout model
Prism render limit	217 series fine; 1,600 crashed while rendering. 625 (25 x 25) untested.
symbol size upper bound	sizes up to 10 render larger; an earlier save normalised 10 -> 8 in the integer mirror